<WA/>
2026

React Life Cycle

Making React Components Alive

Enrico Masala

(Some slides adapted from Fulvio Corno, Luigi De Russis)

CC BY NC SA

https://react.dev/learn/lifecycle-of-reactive-effects

https://react.dev/learn/synchronizing-with-effects

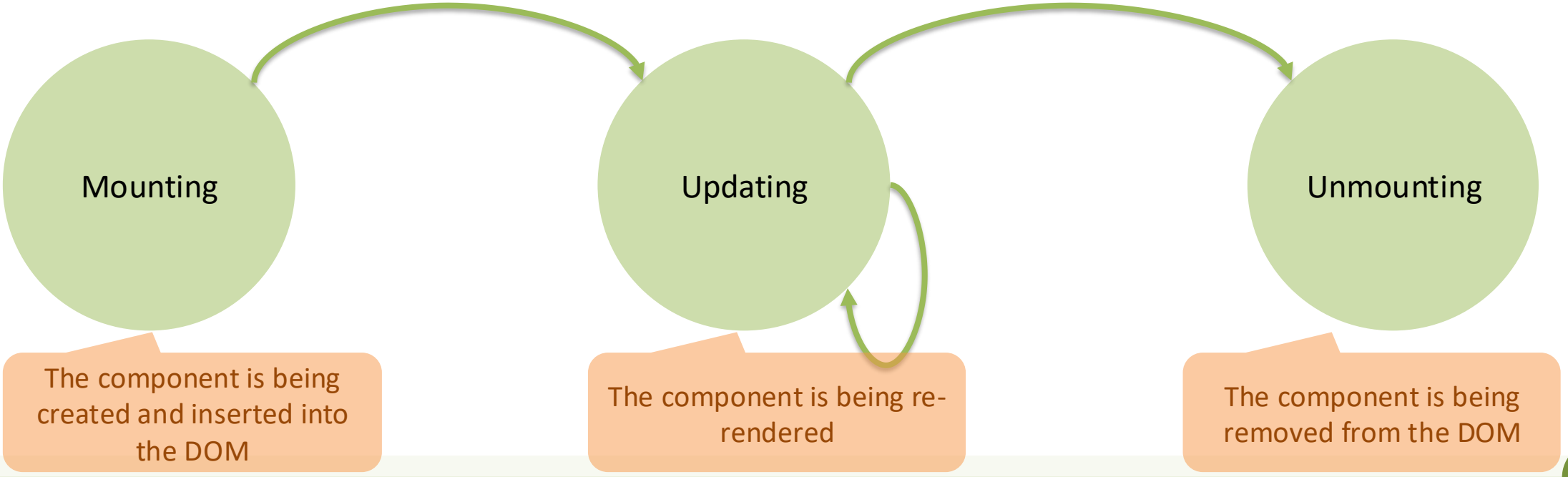
Full Stack React, Chapter “Advanced Component Configuration with props, state, and children”

There’s life before and after `return<JSX>`

COMPONENTS’ LIFECYCLE

Lifecycle Events

- The render action is the most important one for a component
- However, it is also useful to customize what happens at different moments in the evolution of the component



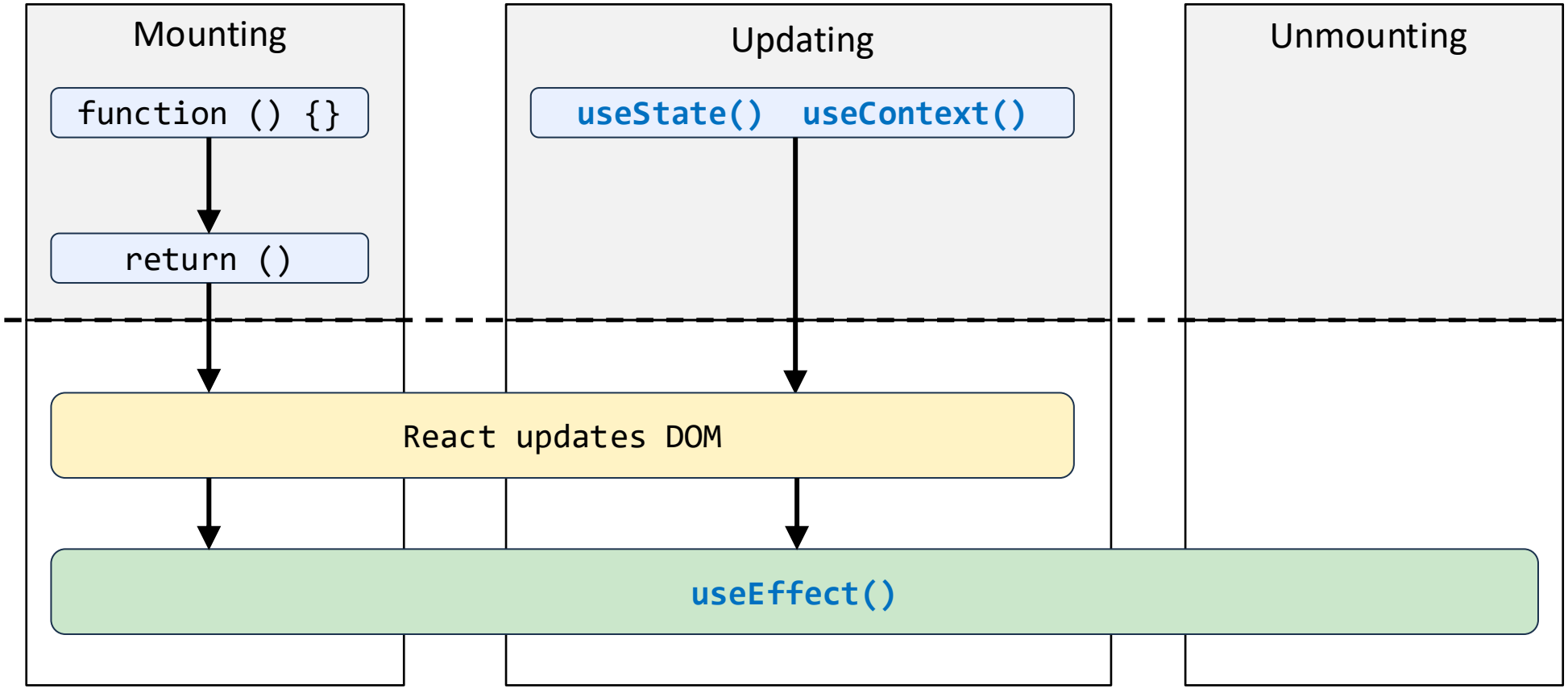
React Hooks Lifecycle

Render phase

Pure, has no side effects. May be paused, aborted or restarted by React

Commit phase

Can work with DOM, run side effects, schedule updates



Side Effects in Function Components

- A functional React component uses props and state to calculate its output
- **Side effect**: any calculation that do not target the output values, anything that affects something *outside the scope of the function component* being executed
- Examples of side effects:
  - **Data fetching**
  - Log recording
  - Setting up a subscriptions (handlers, etc.), or removing them
  - Scheduling additional actions when some state values change
  - Manually changing the DOM in React components
  - Managing timeouts and interval timers
  - …

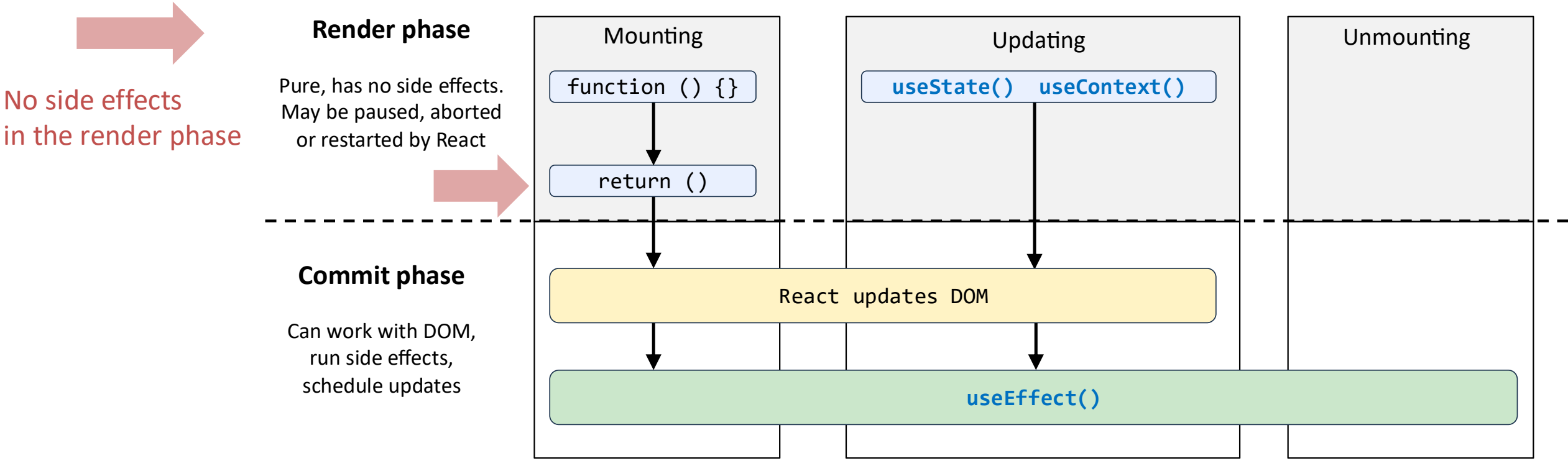
Side Effects in Function Components

- A functional React component uses props and state to calculate its output
- **Side effect**: any calculation that do not target the output values, anything that affects something *outside the scope of the function component* being executed
- Examples of side effects:
  - **Data fetching**
  - Log recording
  - Setting up a subscriptions (handlers, etc.), or r
  - Scheduling additional actions when some stat
  - Manually changing the DOM in React compon
  - Managing timeouts and interval timers
  - …

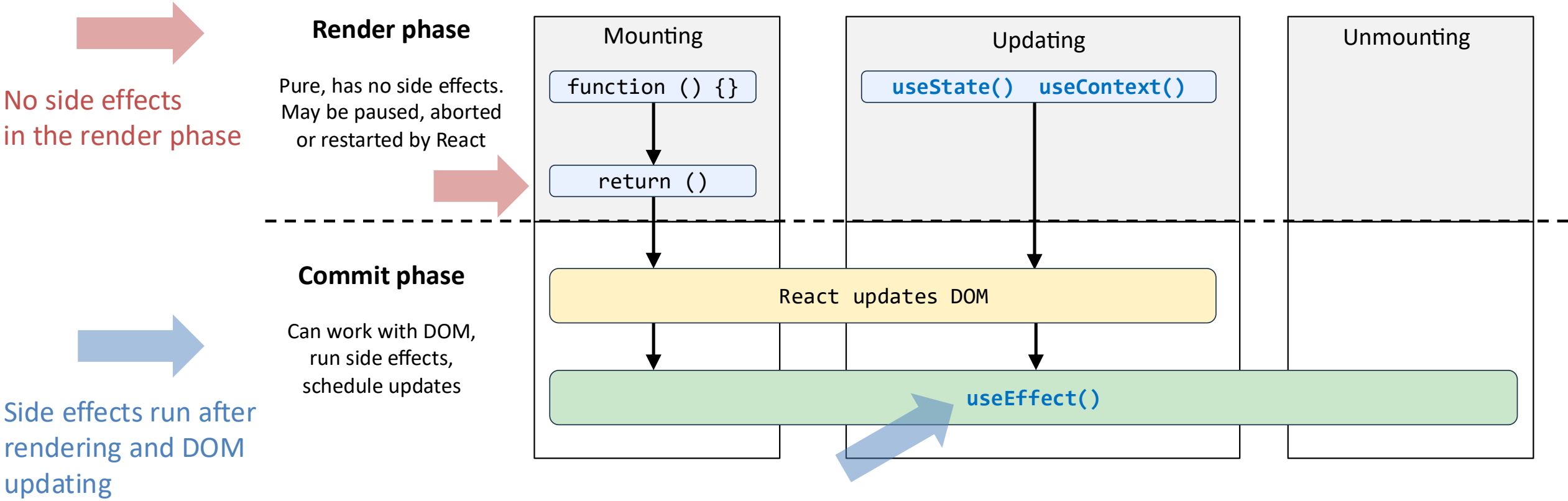
The component rendering and side-effect logic are *independent*.

It would be a mistake to perform side-effects directly in the body of the component.

React Hooks Lifecycle



React Hooks Lifecycle



Full Stack React, “Appendix C: React Hooks”

React Handbook, chapter “Hooks”

https://react.dev/reference/react/useEffect

https://dmitripavlutin.com/react-useeffect-explanation/ (source for many examples)

Side-effects and Life Cycle in Functional Components

USEEFFECT HOOK

No Side Effects in Render Function

```
function GreetBAD(props) {
    const message = `Hello, ${props.name}`;
    const user = {};
    // Calculates some output

    // Bad!
    console.log(`Greetings: ${message}`); // Side-effect!

    // Bad!
    user = API.loadUserInfo(props.userId); // Side-effect!

    // Calculates output
    return <div><p>{message}</p>
    {user.age? <p>Age: {user.age}</p>: null}</div>;
}
```

The side effect will be executed **when React decides to [re-]render**.

Never? Once? Twice? When?

Rendering is **under React control**

Side effects are confined within a useEffect hook.

The hook controls their execution

```
import {useEffect} from "react";

function Greet(props) {
    const message = `Hello, ${props.name}`;
    const user = {};
    // Calculates some output

    useEffect(() => {
        // Good!
        console.log(`Greetings: ${message}`); // Side-effect!
        user = API.loadUserInfo(props.userId); // Side-effect!
    }, []);

    // Calculates output
    return <div><p>{message}</p>
    {user.age? <p>Age: {user.age}</p>: null}</div>;
}
```

How To `useEffect`

Very "dense" API

- `useEffect(callback, [dependencies])`

What to execute

When to execute it

  - `callback`: function containing side-effect logic
  - `useEffect` executes the `callback` function after React has committed the changes to the screen
  - the callback may return a "cleanup function" (see later…)

  - `[dependencies]`: an *optional* array of dependencies
  - `useEffect` executes `callback` only if at least one of the `dependencies` have changed between renderings

`useEffect`'s Dependency Array

- **Not provided**: the side-effect runs after *every* rendering
- **An empty array** `[]`: the side-effect runs *once* after the initial rendering
- **Has props or state values** `[prop1, prop2, ..., state1, state2]`: the side-effect runs *once* after the initial rendering, then only *when any dependency value changes*

`useEffect`'s Dependency Array

- **Not provided**: the side-effect runs after *every* rendering

```
import { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    // Runs after EVERY rendering
  });
}
```

- **An empty array** `[ ]`: the

- **Has props or state value**
  `state2]`: the side-effect runs only *when any dependency value changes*

`useEffect`'s Dependency Array

- **Not provided**: the side-effect runs after *every* rendering

- **An empty array** `[ ]`: the side-effect runs *once* after the initial rendering

- **Has props or state va**
  `state2]`: the side-e *nges*

```
import { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    // Runs ONCE after initial rendering
  }, []);
}
```

`useEffect`'s Dependency Array

- **Not provided**: the side
- **An empty array** `[ ]`: th g

```
import { useEffect, useState } from 'react';

function MyComponent({ prop }) {
  const [state, setState] = useState('');
  useEffect(() => {
    // Runs ONCE after initial rendering
    // and after every rendering ONLY IF `prop` or `state` changes
  }, [prop, state]);
}
```

- **Has props or state values** `[prop1, prop2, ..., state1, state2]`: the side-effect runs only *when any dependency value changes*

Side Effects At Mount Time / Update Time

```
<Count num={num}/> <button onClick={()=>setNum(i=>i+1)}>+</button>
```

```
function Count(props) {
  const isMultiple3 = (props.num % 3 === 0);

  useEffect( ()=>{ console.log(`My static number is ${props.num}`)},  [] ) ;
  // run only once

  useEffect( ()=>{ console.log(`My dynamic number is ${props.num}`)},  [props.num] ) ;
  // run at every change

  useEffect( ()=>{ console.log(`My multiple flag is ${isMultiple3}`)},  [isMultiple3] );
  // run every time isMultiple3 changes

  return <div>{props.num}</div> ;
}
```

Only when the component is *mounted*.

Will print the *initial value* of the num, only.

At mount time, *plus* every time the num changes.

Will print all the values.

At mount time, *plus* every time the `isMultiple3` value changes.

Sometimes it will print, sometimes not.

No "multiple" print here

```
My static number is 3
My dynamic number is 3
My multiple flag is true
My dynamic number is 4
My multiple flag is false
My dynamic number is 5
My dynamic number is 6
My multiple flag is true
My dynamic number is 7
My multiple flag is false
My dynamic number is 8
My dynamic number is 9
My multiple flag is true
My dynamic number is 10
My multiple flag is false
My dynamic number is 11
My dynamic number is 12
...
```

Mount time

Side Effects At Mount Time / Update Time

```
<Count num={num}/> <button onClick={()=>setNum(i=>i+1)}>+</button>
```

```
function Count(props) {
  const isMultiple3 = (props.num % 3 === 0);

  useEffect( ()=>{ console.log(`My static number is ${props.num}`)},  [] ) ;
  // run only once

  useEffect( ()=>{ console.log(`My dynamic
     number is ${props.num}`)},  [props.num] ) ;
  // run at every change

  useEffect( ()=>{ console.log(`My multiple
     flag is ${isMultiple3}`)},  [isMultiple3] );
  // run every time isMultiple3 changes

  return <div>{props.num}</div> ;
}
```

```
My static number is 3
My dynamic number is 3
My multiple flag is true
My dynamic number is 4
My multiple flag is false
My dynamic number is 5
My dynamic number is 6
My multiple flag is true
My dynamic number is 7
My multiple flag is false
My dynamic number is 8
My dynamic number is 9
My multiple flag is true
My dynamic number is 10
My multiple flag is false
My dynamic number is 11
My dynamic number is 12
...
```

TIMELINE

- Component Count is created (num=3) and mounted in App
- Function Count is called
- useEffects are registered (not executed)
- The JSX is returned (with 3)
- Component just mounted => run 1st effect
- Component just mounted => run 2nd effect
- Component just mounted => run 3rd effect
- …
- User clicks, App updates state, num changes to 4
- Function Count is called for re-rendering (num=4)
- The JSX is returned (4)
- `props.num` changed (prev=3, curr=4) => run 2nd effect
- `isMultiple3` changed (prev=`true`, curr=`false`) => run 3rd effect
- …

State with `useEffect`

- A state variable may be listed as a dependency in an effect (as any other value…)
  - When the state changes its value, the effect is run
  - Note: if the state is updated (through setState), but the value does not change, the effect is not run
- Inside a `useEffect` function, you may schedule a state update
  - The state will be updated after the effect is finished (*asynchronously*)
  - If the state value changes, components are re-rendered as usual
  - The state change can be triggered by an async event (e.g., data fetching) *(see later)*

Schedule State Change in useEffect

```
function QuickGate(props) {
    const [open, setOpen] = useState(false) ;

    useEffect(()=>{
        setTimeout(()=>setOpen(false), 500)
    }, [open]);

    const openMe = () => {
        setOpen(true) ;
    };

    return <div onClick={openMe}>
        {open ? <span>GO</span> : <span>STOP</span>}
           </div>;
}
```

TIMELINE

- Component `QuickGate` is created and mounted in App
- Function `QuickGate` is called
- `useState` creates state open with default value
- `useEffect` is registered (not executed)
- The JSX is returned (with: STOP)
- Component just mounted => run effect
  - `setTimeout` is executed: Timeout is set
- Timeout expires
- `setOpen` is executed
- State open becomes `false` => no change
- …
- User clicks
- openMe callback is called
  - `setOpen(true)` executed
- State open becomes `true`
- Component re-renders
- The JSX is returned (with: GO)
- `useEffect` finds open changed (from `false` to `true`)
  - `setTimeout` is executed: Timeout is set
- …
- Timeout expires
  - `setOpen` is executed
- State open becomes `false`
- Component re-renders because of the state change
- `useEffect` finds open changed (from `true` to `false`)
- …

`useEffect` Dependency Array Caveats

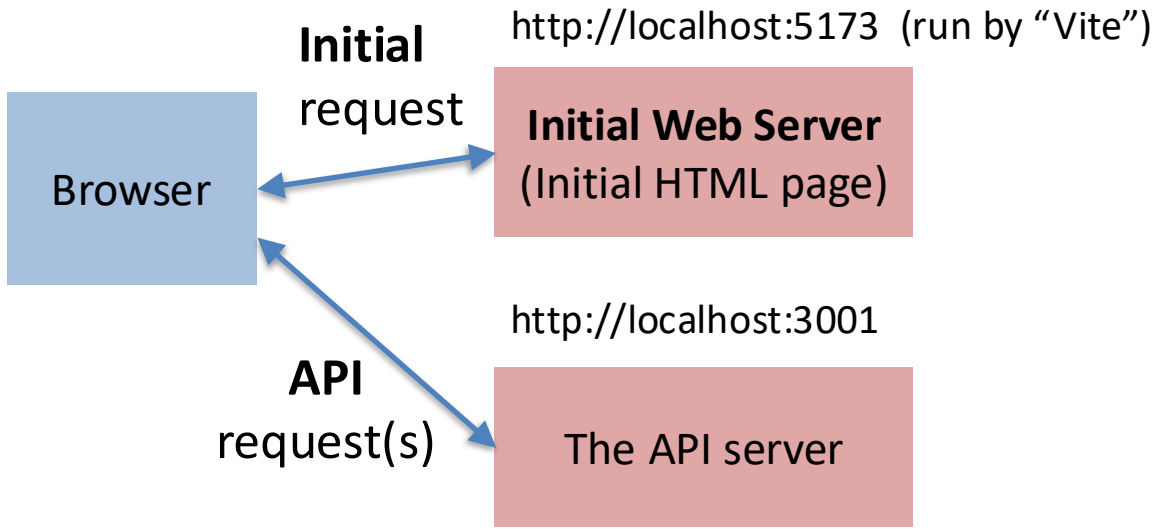
- Make sure the array includes **all** values from the component scope (such as props and state) that change over time and that are used by the effect
- Otherwise, your code will reference stale values from previous renders
  - **Rule**: every value *referenced inside the effect* function should also appear in the dependencies array
    - *arguments* of the functions
    - variables (and functions) accessed through *closure*
- If the array includes variables that *always change* when executing the effect, you risk having an infinite loop

Using `fetch` in `useEffect`

Text: Hello, world

Flipped: pɹoʍ 'olleH

- Example: Type text in a form
- The client sends the text to the server, which returns it in “flipped” form
- The client displays the flipped text below the form



Example

```
import {useEffect, useState} from "react";

function TextFlipper(props) {
    const [text, setText] = useState('');
    const [flipped, setFlipped] = useState('');

    useEffect( ()=>{
        const fetchFlipped = async (reqText) => {
            const response = await fetch('/flip?text='+
                                encodeURIComponent(reqText));
            const responseBody = await response.json();
            setFlipped( responseBody.text );
        };
        fetchFlipped(text);
    }, [text] );

    const handleChange = (ev) => {
       setText(ev.target.value);
    };

    return <div>
        Text: <input type='text' value={text} onChange={handleChange}/><br/>
        Flipped: {flipped}
        </div>;
}
```

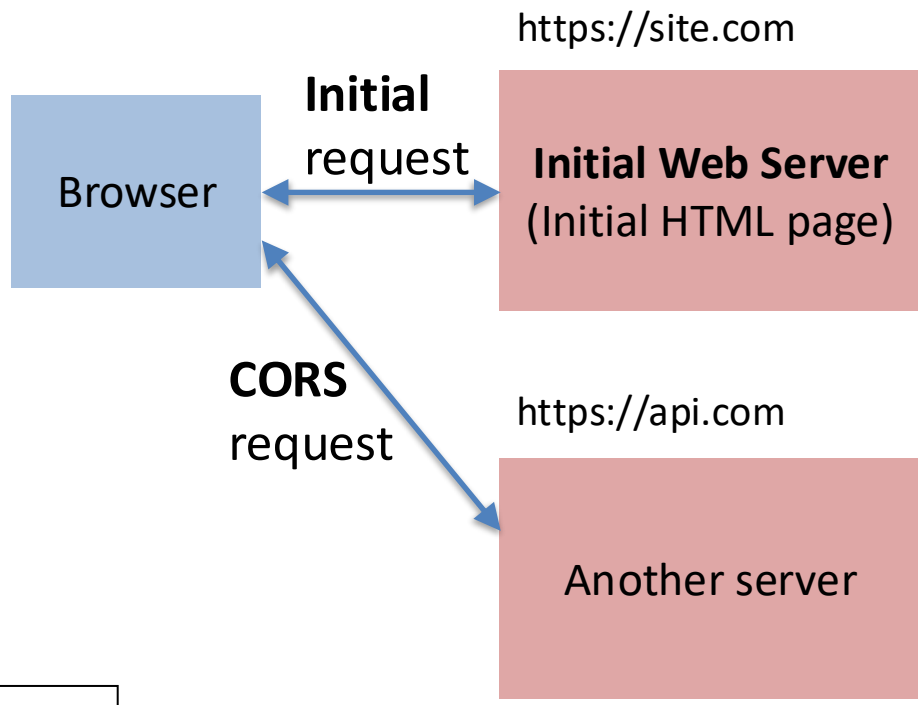
Text: Hello, world

Flipped: pɹoʍ 'olleH

```
import express from 'express';
import flip from 'flip-text';

const app = express() ;

app.get('/flip', (req, res) => {
   const text = req.query.text ;
   const flipped = flip(text) ;
   res.json({text: flipped}) ;
});

app.listen(3001, ()=>{console.log('running')})
```

Note on Using Two Servers

- The browser imposes limitations on what the initial page (in particular Javascript) can do with the second server
  - So called “CORS” request (Cross-Origin Resource Sharing)
  - *More on this soon in the course*
- To make the example (and this situation) work, the API server must allow CORS requests




```
import express from 'express';
import flip from 'flip-text';
import cors from 'cors';

const app = express() ;
app.use(cors());
…
```

`async/await` in useEffect ⚠

- The `callback` argument of useEffect(cb, [...]) **cannot** be an `async` function
- But you can always define and then invoke an `async` function inside the callback itself
  - Inside the async function, you may then use `await`

```
useEffect( ()=>{  //<-- CANNOT be an async function
        const fetchFlipped = async (reqText) => {
            const response = await fetch('/flip?text='+
                                    encodeURIComponent(reqText));
            const responseBody = await response.json();
            setFlipped( responseBody.text );
        };
        fetchFlipped(text); //<-- CAN invoke an async function
    }, [text] );
```

Clean-up After Side Effects

- Some side-effects need cleanup: clear timers, close a socket, …
  - If the callback returns a function, then `useEffect()` considers this as an effect cleanup.
  - After initial rendering, `useEffect()` invokes the callback having the side-effect. `cleanup()` function handler is kept by React but **not** invoked.
  - On later renderings **before invoking** the **next** side-effect callback, `useEffect()` **invokes** the **`cleanup()`** function from the **previous** side-effect execution (to clean up everything after the previous side-effect), *then* runs the current side-effect.
  - Finally, after unmounting the component, `useEffect()` invokes the `cleanup()` function from the latest side-effect.

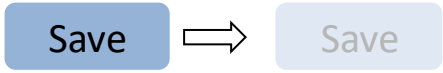
```
useEffect( ()=>{
  // Side-effect

  return function
    cleanup() {
      // Side-effect
        cleanup
    };
  }, [dependencies]);
```

Clean-up After Side Effects

WARNING: frequently asked question at the exam

- At the exam: if the callback returns a function, students must be able to explain why it is needed and the **timeline** of the events
  - Where is the returned function saved?
  - **When this function runs** (see previous slide)
  - Why it is needed in the specific case
  - What can happen if it is not present, i.e., it is not run
  - Recognize which variables (if any) are part of the closure of the function

User Interaction and Slow API Calls

- **Well-designed** application always show a consistent state to the user
  - If they have no data, or if user actions should wait, they tell the user, e.g.:
    - Loading symbols, icons, animations, etc. and/or
    - Disabling parts of the user interface: buttons, fields etc. 

- Usability may be affected
  - For instance: blocking user typing in a form while waiting for API response is bad for usability
  - Revise previous example (flipped text), what if API responses arrive not in order?
    - Fixing it without blocking user input may be difficult
    - Changing user input / interaction method may be an acceptable compromise

User Interaction and Slow API Calls

```
function TextFlipper(props) {
    const [text, setText] = useState('') ;
    const [flipped, setFlipped] = useState('') ;
    const [waiting, setWaiting] = useState(false) ;

    const fetchFlipped = async (reqText) => {
       const response = await fetch('/flip?text='+
                                   encodeURIComponent(reqText));
       const responseBody = await response.json() ;
       setFlipped( responseBody.text ) ;
       setWaiting(false);
    } ;

    const handleChange = (ev) => {
       setText(ev.target.value) ;
    } ;

    const fetchNewFlipped = () => {
       setWaiting(true) ;
       fetchFlipped (text) ;
    } ;

    return <div>
        Text: <input type='text' value={text} onChange={handleChange}/>
              <button disabled={waiting} onClick={fetchNewFlipped} >Update</button><br/>
        Flipped: {waiting && <span>🕘</span>}{flipped}
        </div> ;
}
```

Text: enrico Update

Flipped: ɔɔıuǝ

Text: abcd Update

Flipped: 🕘ɔqɐ

Reasonable compromise for good exam evaluation

- Disabling parts of the user interface (buttons) to avoid sending more than one request at a time
- Marking content as temporary (🕘)

User Interaction and Slow API Calls

```
function TextFlipper(props) {
    const [text, setText] = useState('') ;
    const [flipped, setFlipped] = useState('') ;
    const [waiting, setWaiting] = useState(false) ;

    useEffect( ()=>{
        let ignore = false;
        const fetchFlipped = async (reqText) => {
            const response = await fetch('/flip?text='+ encodeURIComponent(reqText));
            const responseBody = await response.json() ;
            if (!ignore) {
                setFlipped( responseBody.text ) ; // set only if it is the last run
                setWaiting(false);
            }
        };
        setWaiting(true) ;
        fetchFlipped(text) ;
        return () => { ignore = true; };   // runs BEFORE each new useEffect;
                                           // closure on ignore value
    }, [text] ) ;

    const handleChange = (ev) => {  setText(ev.target.value); };

    return <div>
        Text: <input type='text' value={text} onChange={handleChange}/><br/>
        Flipped: {waiting && <span>🕒</span>}{flipped}
        </div> ;
}
```

Note: There are many ignore variables, one for each callback execution

Text: Hello, world

Flipped: pɹoʍ 'olleH

- Now will wait for result of last API call (i.e., last time the useEffect callback was run)
- **Difficult to handle manually** in complex applications
- External/other data loading frameworks may support it in a better way

Not required at exam, just disable actions until response arrives

Inspired from: https://react.dev/learn/you-might-not-need-an-effect#fetching-data

User Interaction and Slow API Calls: Summary

- Find a reasonable compromise between blocking and usability
- Inform the user
- Priority order:
    1. Disabling parts of the user interface: buttons, fields etc. that may lead to situations difficult to handle

       Save ⇨ Save
    2. Show loading symbols, icons, animations, if simple and easy:

Note: `useEffect` Run Twice?

Required at exam

- When Strict Mode is active (i.e., in development), React runs the *setup* and *cleanup* callbacks **one extra time** before the mount of the component.
- This is a **stress-test** that verifies your Effect's logic is implemented correctly.
- If this causes visible issues: your cleanup function is missing some logic. The cleanup function should stop or undo whatever the setup callback was doing.
- **Rule of thumb**: the user should not be able to distinguish between the effect being called once (as in production) and twice (as in development, e.g. React "StrictMode") at mount time

You Might **Not** Need an Effect

Important aspect at exam:
How many useEffects?
Why? Are they necessary?
Alternative possibilities?

- Two common cases where you **do not need** effects:
  1. **To transform data for rendering**. For example, if you want to filter a list in the client before displaying it, you can transform all the data at the top level of your components (so, every time a render is needed), just before returning the JSX.
  2. **To handle user events**. Handle user events in the code of the event handler.
     - For example, the flipText example with "update" button has NO useEffect
- In short: code that runs because a component was displayed (e.g., at mount time) should be in Effects, the rest should/can be in events. Further examples:
  - https://react.dev/learn/you-might-not-need-an-effect

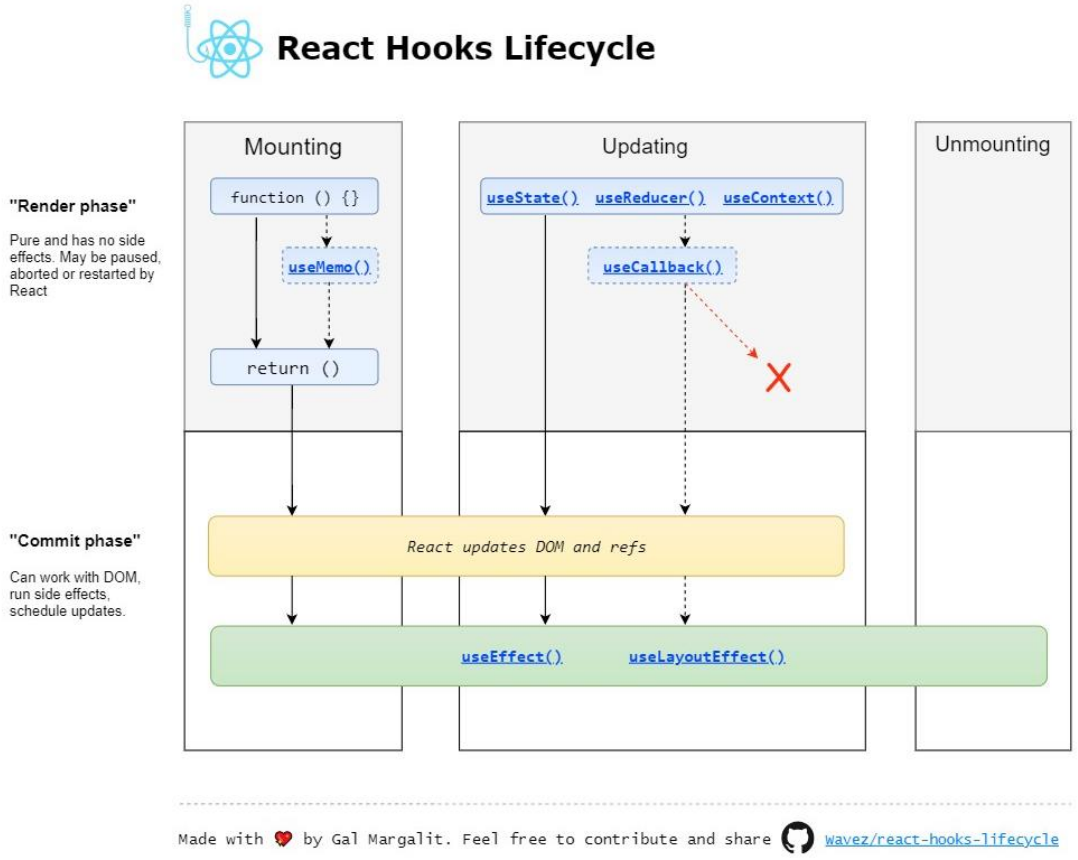
Summary: Four Ways To Call `useEffect`

- Once, when the component mounts
  - `useEffect( () => callOnce(), [] )  // empty 2nd arg`
- On every component render
  - `useEffect( () => callAtEveryRender() ) // missing 2nd arg`
- On every component render, **if** some values changed
  - in addition, it is called when the component mounts
  - `useEffect( () => callIfAnyDepChange(dep1,dep2), [dep1,dep2])`
- When component unmounts
  - `useEffect( () => { doSomething();`
    `return ()=>cleanupFunction(); }, [] )`

https://dev.to/spukas/4-ways-to-useeffect-pf6

How To Handle Other Lifecycle Situations

- Full lifecycle is more complex
- Other hooks available for particular situations
  - `useLayoutEffect`: it fires *synchronously* after all DOM mutations
  - `useMemo`: returns a *memoized* value (re-computed by a pure function when its parameters change)
  - `useCallback`: returns a *memoized* callback function
- *Not recommended in general,*
  <u>DO NOT USE</u> in this course, <u>nor at exam</u>
  (no need)



https://www.robinwieruch.de/react-fetching-data

The Road to Learn React, Chapter “Getting Real with APIs”

Taming the State in React, Chapter “Local State Management”

React as an API Client

HANDLING API CALLS IN REACT

Contents

- Type of states: Application state (permanent, DB), presentation state
- How to integrate remote HTTP APIs: recommendations
- Where/when/how to load data from remote APIs?
- Delays and “loading…”
- Updating remote data
- Typical issues with useEffect, how to avoid them

State Types: Application or Presentation

Application State (or Entity State)

- **Retrieved from the back-end**
- Should **update the back-end**
  - on user-initiated CRUD actions
- Should "periodically" check for updates
  - caused by other users, by other open sessions, or by connected systems
- Globally managed, accessible by various components

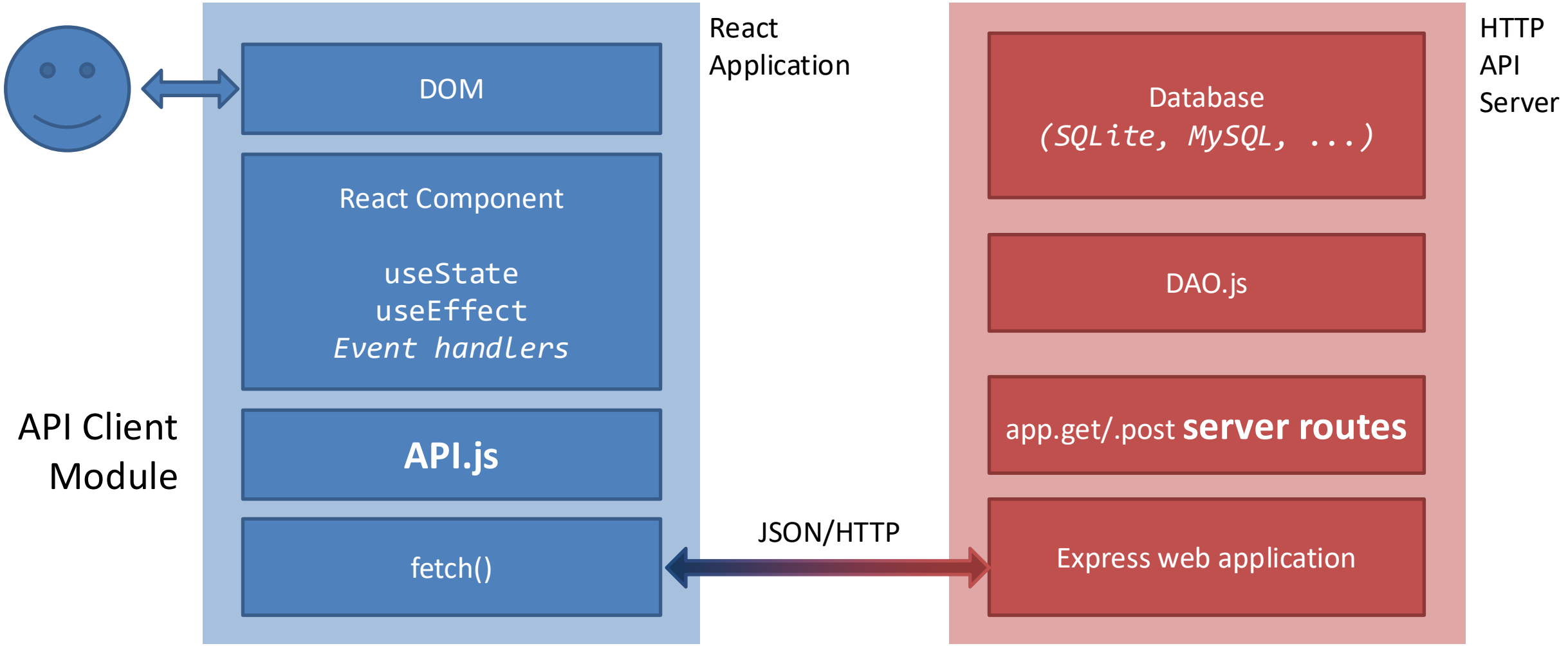
Presentation State (or View State)

- Not stored in the back-end
  - only in React
- Does not need to persist
- Lives and dies within the controlling Component
- Implemented as **Local State**
  - by using useState

Recommendation: API Client Module

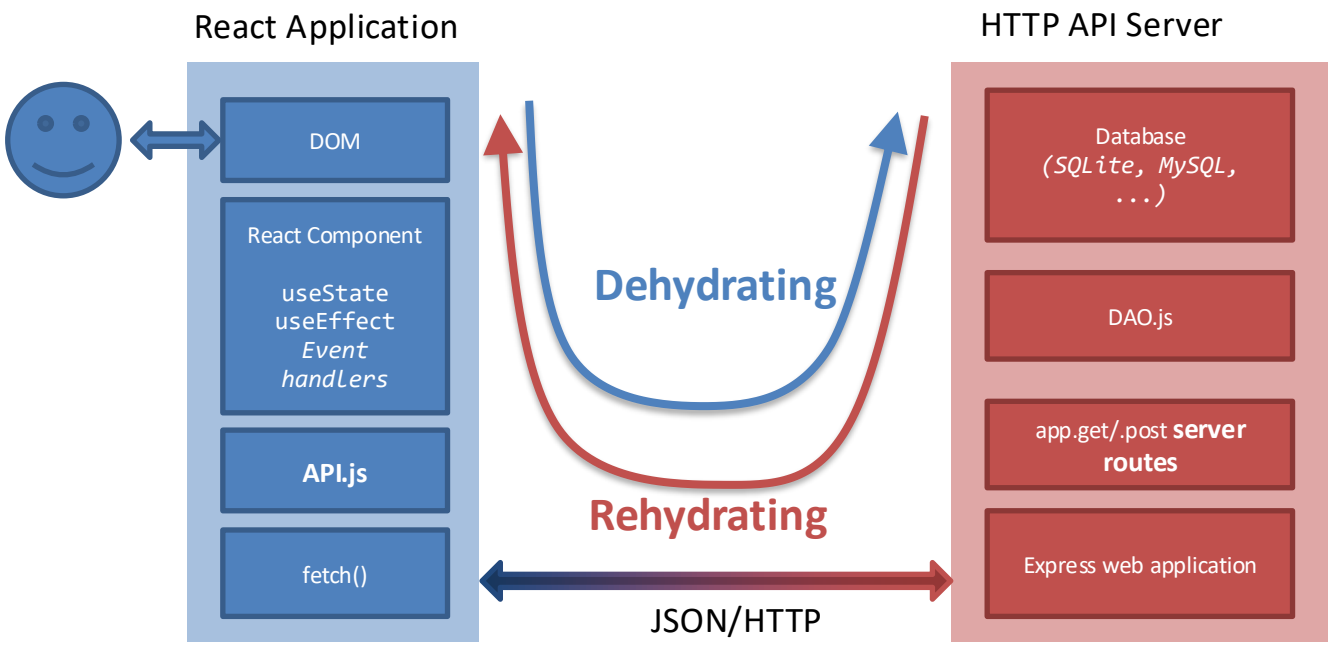
- Keep your fetch methods in a separate JS module (e.g., `API.js`)
- Keeps details of HTTP methods inside the API module
  - API should not depend on React or application state/props
  - Application code should not call fetch directly or have any HTTP information
- Allows easy swapping with “stub” methods for testing

Conceptual Architecture



Rehydrating And Dehydrating

- Application State is retrieved / updated via HTTP APIs
- **Rehydrating** the Application State means getting the state from the server through the HTTP APIs
- **Dehydrating** the Application State means extracting it from the React application and store/update it in the server, through HTTP APIs



Rehydrating And Dehydrating

- **Rehydrating** the Application State:
  - Must happen when the React application mounts
  - **Best place**: inside a `useEffect(fn, [])` method (NB: empty dependency list)
  - Should **also** happen when we want to "refresh" the state: the React app **cannot** know whether ***others*** changed the info provided by the API server
- **Dehydrating** the Application State:
  - May happen several times during the React app execution
  - Should happen <u>whenever</u> something (in the Application State) is modified
    - Added items, updated items, deleted items, booking an item, bid for something, in general request server actions, etc.

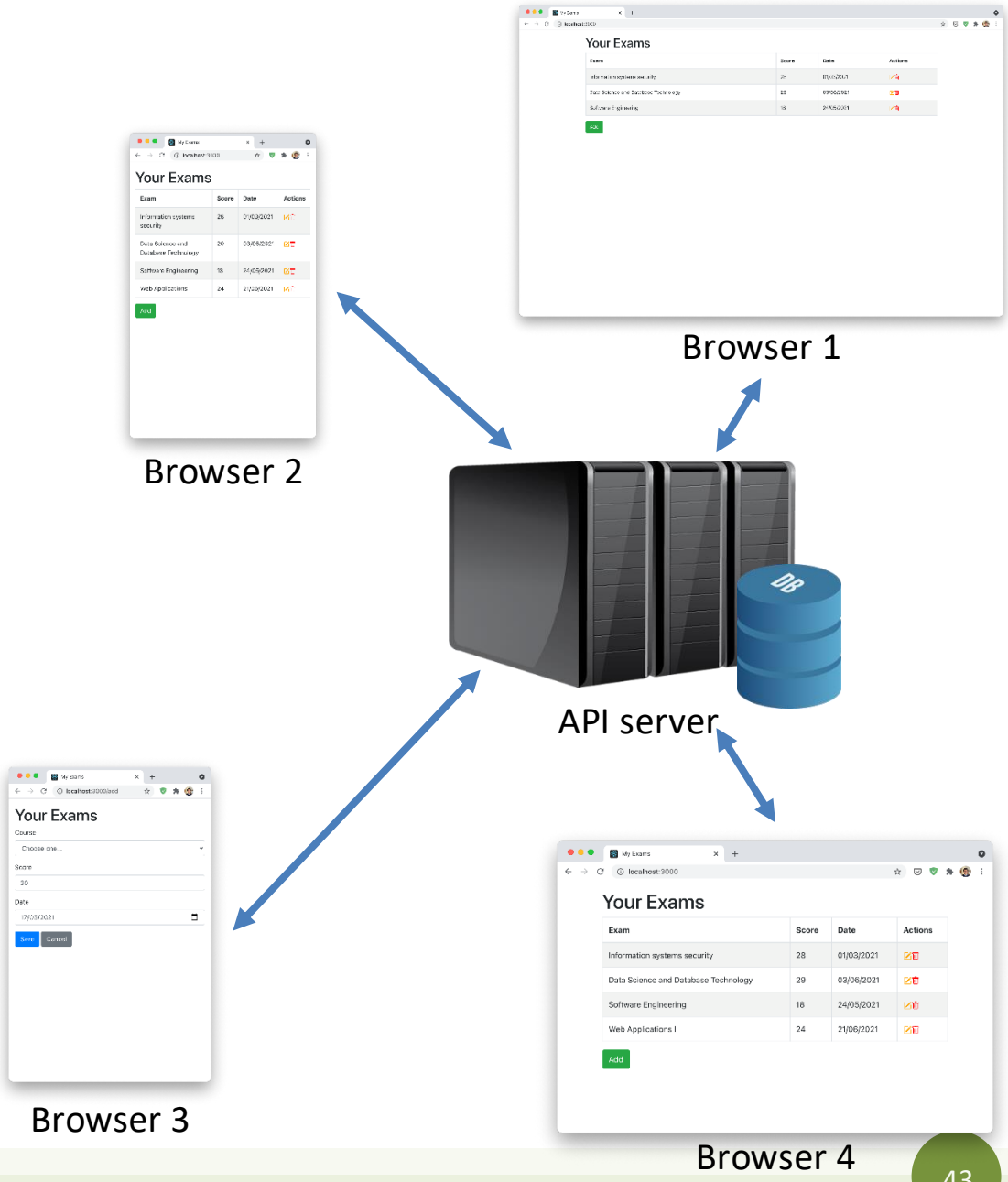
Example: Rehydrating At Mount Time

- Very similar to what we saw before…
- Rehydrating may require some time, while the component renders “empty” (with the initial state)
  - May need to use the approach for *handling slow responses*
  - i.e., a loading/waiting **local** state

```
import { useEffect, useState } from 'react';

function ShoppingList() {
  const [list, setList] = useState([]);
  const [loading, setLoading] = useState(true) ;

  useEffect(()=> {
    const getItems = async () => {
      const response = await fetch('/api/items');
      const items = await response.json();
      setList(items);
      setLoading(false);
    };
    getItems();
  }, []);

return (<>
  {loading && <span>🕘</span>}
  <ul>{list.map((item, i) =>
    <li key={i}>{item}</li>)}</ul>
  </>);
}
```

The “N-Clients Problem”

- Note that we are creating a web application that will be opened by *multiple* browsers / users at once
  - They all read and write info from a *unique* API server, however
- What happens in the web app running in, e.g., Browser 1 when Browser 3 updates something in the API server?
  - How can one web app know that *someone else* changed *something* in the server?



The “N-Clients Problem”

The Real Solution

- The *server* communicates changes as soon as they appear
  - to *all* the current consumers of its information
- **Out of scope** for this course
  - unfortunately!
- For the curious:
  - WebSockets (e.g., https://socket.io)
  - PubSub (publish/subscribe) mechanisms

The Better-Than-Nothing Solution

- The *web app* asks for data as frequently as possible (i.e. it refreshes the state)
  - when it loads a new page/view
  - after adding/updating/removing something
  - periodically (i.e., polling) if appropriate
  - …
- **Not a solution**: just a way to *minimize* the problem
  - Do not periodically query the server unless explicitly required

Rehydrating To Refresh The State

- Once you know that something should have changed in the API server, because of **dehydration operation** (e.g., create/update/delete operation):
  - It is the result of code running in the client (e.g., in event handler)
  - Simply run/call `API/fetch` to refresh the data
    - Note: no need for useEffect here
- Once you access a new view that needs data not yet loaded or wants a refreshed view of the data:
  - Typically, it happens that a new React component is mounted
  - Write `useEffect` that run a callback at component mount time
  - Include an appropriate list of dependencies if necessary (e.g., props, and/or parameters from a parametric URL, etc.)

Dehydrating During Updates: Approaches

Bare minimum for the exam

```
const addItem = async () => {
  setList(items => [...items, element]);

  const response = await fetch('/api/items', {
    method: 'POST', body: element,
  });
  ...
};

return (...
    <input type="text" value={element} ... } />
    <button onClick={addItem}>Add</button>
  ...);
```

The two updates (remote API, local state) run **in parallel**. **Optimistic state update**: it assumes that remote state will be updated without errors => **Risky!**

Do **NOT** use as it is for the exam

```
const addItem = async () => {
  const response = await fetch('/api/items', {
    method: 'POST', body: element,
  });
  if (response.ok)
    setList(items => [...items, element]);
  ...
};

return (...
    <input type="text" value={element} ... } />
    <button onClick={addItem}>Add</button>
  ...);
```

The state is updated only **after** checking that the request is successfully => **No parallel updates!**
Issue: user **may <u>not</u> see** the added item for a while…

Compromise #1: Block User Until Response Arrives

```
const [waiting,setWaiting] = useState(false);
const addItem = async () => {
  setWaiting(true);
  const response = await fetch('/api/items', {
    method: 'POST', body: element,
  });
  if (response.ok) {
    // Rehydrating to refresh state
    const newItemList = await fetch('/api/items');
    setList(newItemList);
    setWaiting(false);
  }  ...
};

return (...
    <input type="text" value={element} ... } />
    <button disable={waiting} onClick={addItem}>
       {waiting?'Adding...':'Add'}</button>   ...);
```

1. **Use** a waiting state to record that updates are in progress
2. **Show** this state to the user
   - e.g., by a message (“adding…”)
3. **Block** the interface from performing additional operations of the same type until the response is back
   - e.g., by disabling a button, …
4. **Refresh** the *entire state* of the component as soon as the server completes the update operation
   - Note: **NO need for useEffect**

Compromise #2: Optimistic, Warn the User

```
const addItem = async () => {
  setList(items => [...items, `${element} (temp)`]);

  const response = await fetch('/api/items', {
    method: 'POST', body: element,
  });
  if (response.ok) {
    // Rehydrating to refresh state
    const newItemList = await fetch('/api/items');
    setList(newItemList);
  }
  ...
};

return (...
    <input type="text" value={element} ... } />
    <button onClick={addItem}>Add</button>
  ...);
```

1. **Update** the state in parallel so that the user can see that something happened
2. **Mark** the just updated item as *temporary/in progress*
   - e.g., by using a different background color, label, … than the others
3. **Refresh** the *entire state* of the component as soon as the server completes the update operation
   - Note: **NO need for useEffect**

- May become complex if user can initiate other actions without waiting for the response

Which one?

- Depends on the application and users (e.g., is blocking some user interaction acceptable?), consequences when update is not successful (losing work done on not-yet-updated server state), etc.

Blocking: in general enough for good exam result

- **Blocking:** ok if the user is expected to wait for the result of the operation
  - For instance: payments, other application-level updates one should wait for
- **Optimistic update:** ok if the user does not immediately need to work on (e.g., modify) the new state, user can operate on other data in the meanwhile, etc.
  - E.g., fill-in autosave content, delete, …
- **Both approaches** (if appropriate / if makes sense)

Failure due to “N-Clients Problem”

- Operations may fail due to others interacting with the server. Examples:
  - Create: another object with the same key has been created in the meanwhile
  - Update: the object has been deleted/modified in the meanwhile
  - Delete: the object has already been deleted (e.g. by others)
- Be prepared to handle concurrency failures! Good web apps **<u>require</u>** handling concurrency failures!
- How to handle failures:
  1. Tell the user about failure in a way that can be easily understood
  2. Dehydrate to refresh client application state
  3. If appropriate, let the user perform the operation again on the updated data
- <u>Do not</u> try to avoid the issue by a periodical time-based refresh: it does not scale and in any case it does not provide any guarantee

High impact on the final exam grade

“N-Clients Problem” and APIs (recall)

- Web applications are **multi-user and concurrent by definition**
- All server actions (=state changes) that may have different output depending on others’ actions should be requested using a **single API call**
- Examples: bid in an auction, increment a counter, resource reservation (places/seats), purchase operations, applying discounts (avoid applying it multiple times) ...
- Design a **single API** that implement the requested operation in a single HTTP transaction, ending with **either success or failure**
- Note that the client does not know how the server implements the requested functionality, but it must handle **BOTH** SUCCESS and FAILURE appropriately

High impact on the final exam grade

“N-Clients Problem” and APIs (recall)

- API examples

Major source of errors at exam

```
GET /bestOffer
HTTP response body: 100
... potential actions by others!
PUT /bestOffer
HTTP request body: 105
```

state may change in the meanwhile due to others!

```
POST /newOffer
HTTP request body: 105
HTTP response body: {status: 'accepted', bestOffer: 105}
```

```
POST /newOffer
HTTP request body: 105
HTTP response body: {status: 'rejected', bestOffer: 110}
```

new server state, update client state

```
POST /addLike
```

```
POST /bookSeats
HTTP request body: [ A2, B3, B4 ]
```

```
POST /buyItems
HTTP request body:
[ {itemId: 43, qty: 3},
{itemId: 88, qty: 1} ]
```

When `useEffect` may be Useful for Data Loading

```
import { useEffect, useState } from 'react';

function FetchEmployeesByQuery({ query }) {
  const [employees, setEmployees] = useState([]);

  useEffect(() => {
    async function fetchEmployees() {
      const response = await fetch(
            `/employees?q=${encodeURIComponent(query)}`
      );
      const fetchedEmployees = await response.json();
      setEmployees(fetchedEmployees);
    }
    fetchEmployees();
  }, [query]);

  return (
    <div>
      {employees.map(name => <div>{name}</div>)}
    </div>
  );
}
```

- `useEffect()` can perform data fetching side-effect
- Component is not to be re-mounted (it is still displayed) but it receives a prop (or a part of the URL) that may change
  - Of course, fetch runs also at first mount
- When the response is available, the employees state is updated
  - Component will re-render

Typical Errors with `useEffect`: Infinite Loops

- One of the main pitfalls that might happen with `useEffect`
  - infinite loops both in rendering and due to external (e.g., HTTP) calls (that set a state)
  - especially when `useEffect` is used with `useState`

- Two significant cases:
  1. The dependency array is missing, but it should not
  2. One or more items in the dependency array are a JavaScript Object `{}` or Array `[]` *(Tip: avoid it!)*

Examples from: https://dmitripavlutin.com/react-useeffect-infinite-loop/

Example: Missing Dependencies

What is wrong, here?

```
import { useEffect, useState } from 'react';

function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);

  useEffect( () => setCount((c) => (c + 1)) );

  const handleChange = (ev) => setValue(ev.target.value);

  return (
    <div>
      <input type="text" value={value} onChange={handleChange} />
      <div>Number of changes: {count}</div>
    </div> );
}
```

1. Set Up Dependencies Correctly

- Without the dependency (no dependency array), the code in the example will re-render the component **forever**
- It is also a clear error:
  - the update of `count` depends on the change of `value`

```
import { useEffect, useState } from 'react';

function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);

  useEffect( () => setCount((c) => (c + 1)), [value] );

  const handleChange = ({ target }) => setValue(target.value);

  return (
    <div>
      <input type="text" value={value} onChange={handleChange} />
      <div>Number of changes: {count}</div>
    </div> );

}
```

Example: Objects as Dependencies

What is wrong, here?

```
function CountSecrets() {
  const [secret, setSecret] = useState({ value: "", countSecrets: 0 });

  useEffect(() => {
    if (secret.value === 'secret')
      setSecret(s => ({...s, countSecrets: s.countSecrets + 1}));
    }, [secret]);

  const onChange = (ev) => { setSecret(s => ({ ...s, value: ev.target.value })); };

  return ( <div>
    <input type="text" value={secret.value} onChange={onChange} />
    <div>Number of secrets: {secret.countSecrets}</div>
    </div> );
}
```

2a. Avoid Objects As Dependencies

- **Problem**: `secret` as a dependency!
- Inside `useEffect`, when the input value equals 'secret', `setSecret()` is called
- `setSecret()` increments the secrets counter, but also creates a new object
  - `secret` is now a new object, and the dependency has changed
- So `useEffect` invokes again the callback that updates the state, and a new `secret` object is created again, etc.
- **How to solve:**
  DO NOT use objects as dependencies!

```
import { useEffect, useState } from 'react';

function CountSecrets() {
  const [secret, setSecret] = useState(
                                { value: "", countSecrets: 0 });

  useEffect(() => {
     if (secret.value === 'secret')
      setSecret(s => ({...s, countSecrets: s.countSecrets + 1}));
    }, [secret.value]);

  const handleChange = ({ target }) => { setSecret(
                        s => ({ ...s, value: target.value })); };

  return ( <div>
    <input type="text" value={secret.value}
                        onChange={handleChange} />
    <div>Number of secrets: {secret.countSecrets}</div>
    </div> );
}
```

2b. Avoid Arrays As Dependencies

```
import { useEffect, useState } from 'react';

function ShoppingList() {
  const [list, setList] = useState([]);

  useEffect(()=> {
    const getItems = async () => {
      const response = await fetch('/api/items');
      const items = await response.json();
      setList(items);
    };
    getItems();
  }, [ ]); // don't use: [list]

  return (
    <ul>{list.map((item, i) =>
         <li key={i}>{item}</li>)}</ul> );
}
```

- The same issue might happen with arrays…
- … so, it is better to avoid arrays as dependencies as well
  - you can use an empty dependency array [ ]
  - or any item in the array, e.g., the `length` property (if appropriate)
  - or an additional state to trigger `useEffect` *(avoid if possible, lead to complex state dependencies)*

Full Stack React, “Appendix C: React Hooks”

React Handbook, chapter “Hooks”

https://react.dev/reference/rules/rules-of-hooks

Peeking Under the Hood

THE RULES OF HOOKS

Quiz

- What is the “magic” behind `useState`?
- How can the same function return different state variables?
- How can the values be persisted across function calls?

```
function Example(props) {

  [hidden, setHidden] = useState(true) ;
  [count, setCount] = useState(0) ;
  [mode, setMode] = useState('view') ;

  . . .

  setHidden(false) ;
  . . .
  setCount( c => c+1 ) ;
  . . .
  setMode('edit') ;
  . . .

}
```

Answer

- React associates an array of Hook “slots” to each functional component
  - Slots are stored with the function, therefore they are persistent
- Each time you call a Hook, a new “slot” is used
  - The first time, it is created
  - The other times, it is reused

```
function Example(props) {

  [hidden, setHidden] = useState(true) ;
  [count, setCount] = useState(0) ;
  [mode, setMode] = useState('view') ;

  . . .

  setHidden(false) ;
  . . .
  setCount( c => c+1 ) ;
  . . .
  setMode('edit') ;
  . . .

}
```

Corollary

- React must “know” which functions may host Hooks
- Hooks must always be called in the same order each time a component renders

```
function Example(props) {

  [hidden, setHidden] = useState(true) ;
  [count, setCount] = useState(0) ;
  [mode, setMode] = useState('view') ;

  . . .

  setHidden(false) ;
  . . .
  setCount( c => c+1 ) ;
  . . .
  setMode('edit') ;
  . . .

}
```

Hook Usage Rules

- Only Call Hooks at the Top Level
  - Always call Hooks at the top level of your React function
  - Do not call Hooks inside loops, conditions, or nested functions
- Only Call Hooks from React Functions
  - Do not call Hooks from regular JavaScript functions
  - You may call Hooks from React function components
  - You may call Hooks from custom Hooks *(not discussed in this course)*

https://react.dev/reference/rules/rules-of-hooks

License

- These slides are distributed under a Creative Commons license "**Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)**"
- **You are free to:**
  - **Share** — copy and redistribute the material in any medium or format
  - **Adapt** — remix, transform, and build upon the material
  - The licensor cannot revoke these freedoms as long as you follow the license terms.
- **Under the following terms:**
  - **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
  - **NonCommercial** — You may not use the material for commercial purposes.
  - **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.
  - **No additional restrictions** — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.
- https://creativecommons.org/licenses/by-nc-sa/4.0/